

Идея артефактов.

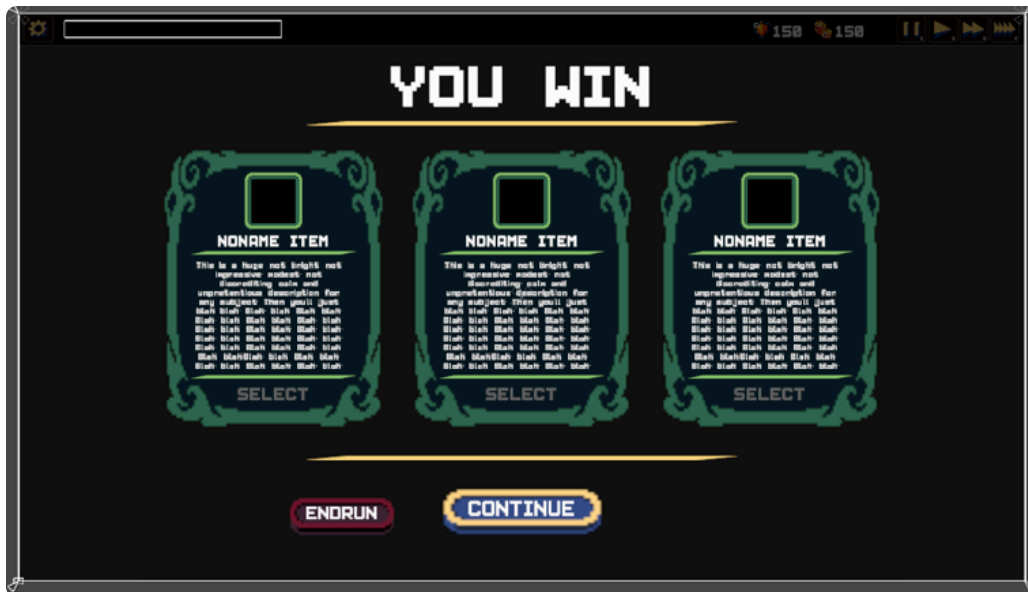
- В бесконечный TD хочется добавить что-то, что могло бы сделать игру веселее, при этом не позволяя игроку прогрессировать в пассивном плане. Если просто увеличивать силу мобов и силу игрока, то игра не будет становиться сложнее, ведь состояние сил всегда будет одинаково. Если позволять расти только силе врагов, то рано или поздно они станут очень сильными. Относительно простенькая система градации сложности предполагает, что игрок проиграет в любом случае, однако поражение (превосходство сил врага) можно оттянуть через специальные предметы. Вот только хотелось бы достичь такой специфики предметов, которая не просто бы улучшала характеристики, а делала игру разнообразнее и веселее, учитывая, что я всё-таки вряд ли смогу в хороший баланс.

Как и во всех рогаляках - предметы почти всегда падают именно после завершения этажа \ комнаты \ волны. Когда игрок преодалеет все волны и локация будет завершена, ему на выбор будет выдано три предмета, из которых он может взять лишь один.

NextFloorScreen (ArtefactsScreen).

- Если же игрок пережил все волны, то, в духе рогаляков, нам открывается меню с выбором одного из нескольких предметов. Здесь можно покинуть игру или продолжить. Однако не выбрав предмета - продолжить не получится.

Три бокса с описанием, иконкой и названием предмета - классика. При выборе предмета кнопка SELECT меняется на SELECTED и перекрашивается в зелёный. При выборе другого → прошлая возвращается в исходное состояние, а новая приобретает эффект выбора.



Интерфейс выбора предметов.



Выбранный предмет меняет надпись и подсвечиваемость.

ItemPanel.

```

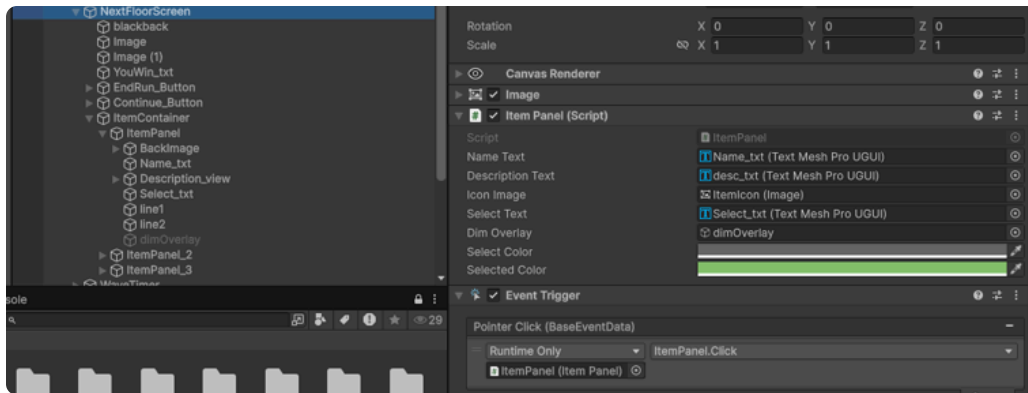
1 using System;
2 using TMPro;
3 using UnityEngine;
4 using UnityEngine.UI;
5
6 public class ItemPanel : MonoBehaviour
7 {
8     public Action<ItemPanel> OnClicked;
9
10    [SerializeField] private TMP_Text nameText;
11    [SerializeField] private TMP_Text descriptionText;
12    [SerializeField] private Image iconImage;
13    [SerializeField] private TMP_Text selectText;

```

```

14 [SerializeField] private GameObject dimOverlay;
15
16 [SerializeField] private Color selectColor;
17 [SerializeField] private Color selectedColor;
18
19 private ArtefactSO artefact;
20
21 public void Initialize(ArtefactSO data)
22 {
23     gameObject.SetActive(true);
24     artefact = data;
25     nameText.text = data.name;
26     descriptionText.text = data.description;
27     iconImage.sprite = data.icon;
28
29     SetSelected(false);
30     dimOverlay.SetActive(false);
31 }
32
33 // Инициализируем наш интерфейс данными из пришедшего пакета данных.
34
35 public void Click()
36 {
37     OnClicked?.Invoke(this);
38 }
39
40 // Вызывает событие клика. На родительском объекте панели висит InventTr
41 // компонент, который обрабатывает клик и вызывает метод Click().
42 // К слову, событие будет обрабатывать отдельный класс, он будет и вызы
43 // SetSelected();
44
45 public void SetSelected(bool selected)
46 {
47     selectText.text = selected ? "SELECTED" : "SELECT";
48     selectText.color = selected ? selectedColor : selectColor;
49     dimOverlay.SetActive(!selected);
50 }
51
52 // SetSelected() - эффект выбора панели. Меняет цвет и надпись,
53 // а также включает затемнение.
54
55 public ArtefactSO GetArtefact() => artefact;
56
57 // GetArtefact() - возвращает сам SO артефакт. Понадобиться, когда игро
58 // будет выбирать панель и нажимать продолжить, чтобы применить артефак
59
60 public void Clear()
61 {
62     nameText.text = "";
63     descriptionText.text = "";
64     iconImage.sprite = null;
65     SetSelected(false);
66     selectText.text = "SELECT";
67     selectText.color = selectColor;
68     dimOverlay.SetActive(false);
69     gameObject.SetActive(false);
70 }
71
72 // Clear() - сбрасывает интерфейс и поле артефакта, чтобы повесить след
73
74 }

```



Иерархия и инспектор.

ArtefactSO.

Создаём ScriptableObject класс, который будет конфигом для артефакта. А также сам эффект тоже делаем SO. Это позволит в артефакт пробрасывать SO настраиваемый, самостоятельный вариант эффекта.

```

1 using UnityEngine;
2
3 [CreateAssetMenu(menuName = "Artefacts/Artefact")]
4 public class ArtefactSO : ScriptableObject
5 {
6     public int IDA;
7     public string artefactName;
8     [TextArea] public string description;
9     public Sprite icon;
10
11     public ArtefactEffectSO effectArtefact;
12 }
13
14 public abstract class ArtefactEffectSO : ScriptableObject
15 {
16     public abstract void Equip();
17     public abstract void Dequip();
18 }

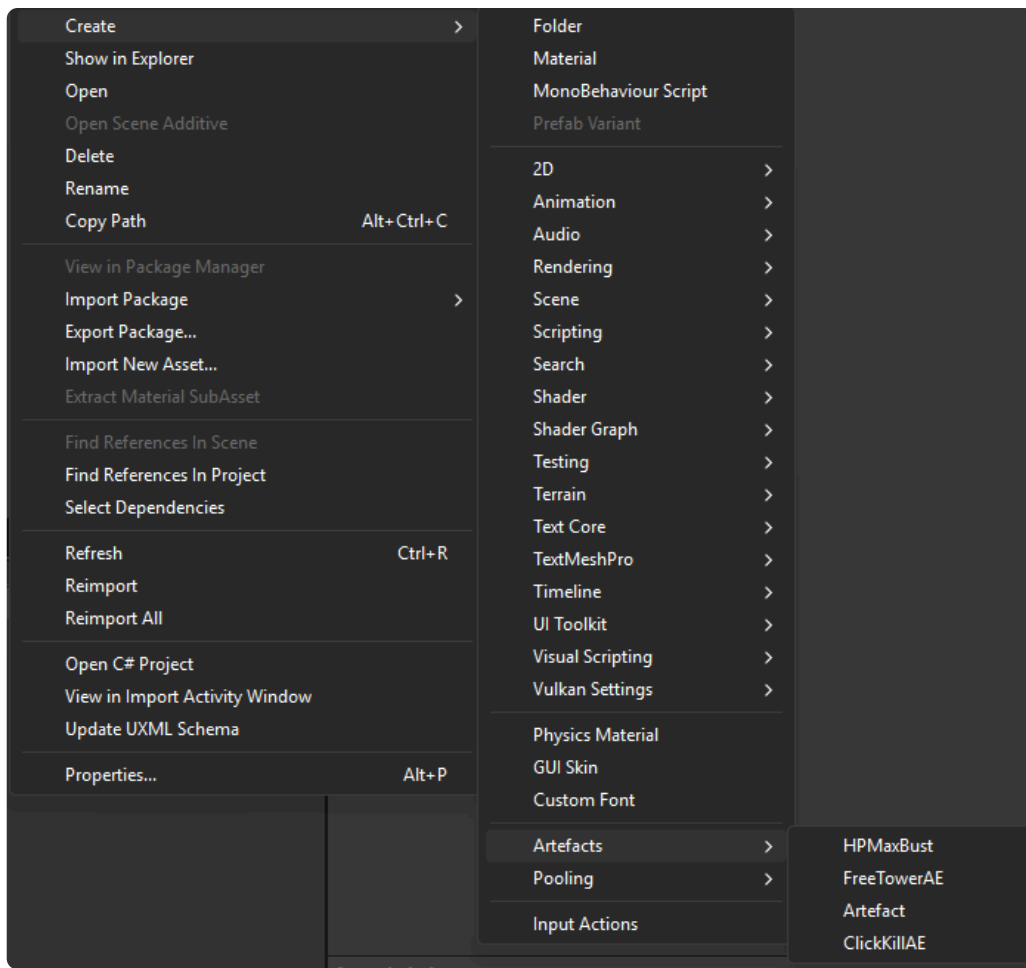
```

Пример предмета:

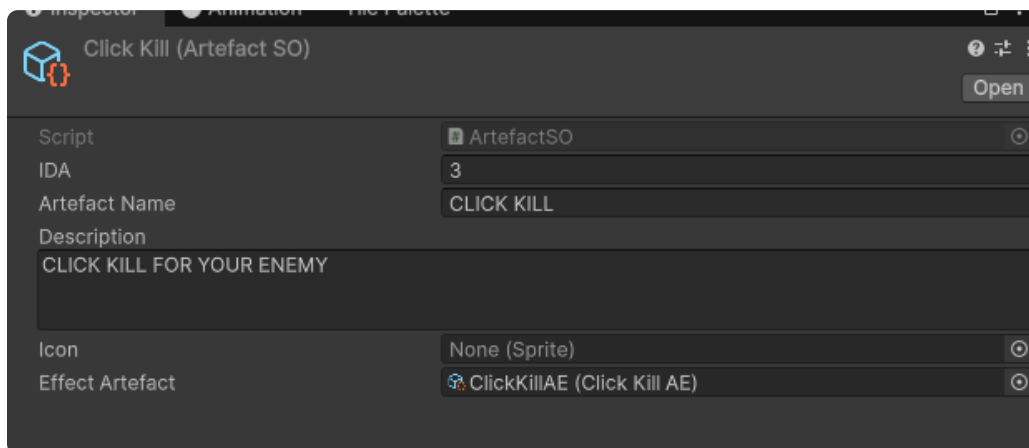
```

1 using UnityEngine;
2
3 [CreateAssetMenu(menuName = "Artefacts/ClickKillAE")]
4 public class ClickKillAE : ArtefactEffectSO
5 {
6     public override void Dequip()
7     {
8         Debug.LogError("Artefact 3 Dequip");
9     }
10
11     public override void Equip()
12     {
13         Debug.LogError("Artefact 3 Equip");
14     }
15 }

```



Создание SO эффекта.



Настройка конфига артефакта.

NextFloorScreen.

```

1 using System;
2 using System.Collections.Generic;
3 using UnityEngine;
4 using UnityEngine.UI;
5
6 public class NextFloorScreen : MonoBehaviour
7 {
8     public Action OnEndRunWinMenu;
9     public Button continueButton;

```

```

10
11 [SerializeField] private List<ItemPanel> itemPanels;
12 [SerializeField] private int itemAmount = 3;
13 public int GetItemAmount() => itemAmount;
14 public void SetItemAmount(int amount) => itemAmount = amount;
15
16 private ItemPanel selectedPanel;
17 private ArtefactSO selectedArtefactSO;
18
19
20 private void Awake()
21 {
22     Hide();
23     foreach (var itemPanel in itemPanels)
24     {
25         itemPanel.OnClicked += ClickedPanel;
26     }
27 }
28
29 // Храним лист панелей и выбранный артефакт. Скрываем сами себя и подп
30 // под событие каждой панели метод ClickedPanel();
31
32 public void Show()
33 {
34     gameObject.SetActive(true);
35     LoadNewItems();
36 }
37
38 // Show() - предполагает включение и загрузку случайных предметов в па
39 // Мы просто будем пробрасывать полученные ArtefactSO внутрь
40 // ItemPanel.Initialize(ArtefactSO) и выводить на экран.
41
42 public void Hide() =>
43     gameObject.SetActive(false);
44
45 public void ContinueButton()
46 {
47     if (continueButton == null) return;
48     ArtefactManager.Instance.EquipArtefact(selectedArtefactSO);
49     GameFlowManager.Instance.ContinueAfterArtifact();
50 }
51
52 // ContinueButton() - вызывается нажатием кнопки на интерфейсе.
53 // Передаёт в ArtefactManager выбранный предмет и вызывает продолжение
54 // у GameFlowManager.
55
56 // Интересный момент - проверки на то, что предмет выбран - нет.
57 // Хотя и можно было обойтись флагом, решением стало просто сделать кн
58 // некликабельной. А как только выбирается хотя бы один предмет (клик
59 // по панели вызывает событие и событие вызывает ClickedPanel(передава
60 // - то кнопка становится активной. А вернуться к состоянию отсутствия
61 // выбора нельзя.
62
63 public void EndRunButton() =>
64     GameFlowManager.Instance.Defeat();
65
66 public void LoadNewItems()
67 {
68     ClearPanels();
69     var items = ArtefactManager.Instance.GetRandomArtefacts(itemAm
70
71     int count = Mathf.Min(items.Count, itemPanels.Count);
72     if (count < 1) {
73         Debug.LogError("Not a single item was received from the Arte
74     }
75     for (int i = 0; i < count; i++)
76     {

```

```

77         itemPanels[i].Initialize(items[i]);
78     }
79 }
80
81 // LoadNewItem() - очищаем панели, получаем список случайных предметов
82 // менеджера, проверяем на меньшее значение. И инициализируем присутст
83 // панели. В случае если предметов будет меньше панелей, то панели не
84 // инициализируются и не включаются.
85
86 private void ClearPanels()
87 {
88     foreach (var item in itemPanels)
89     {
90         item.Clear();
91     }
92     selectedPanel = null;
93     selectedArtefactSO = null;
94     continueButton.interactable = false;
95 }
96
97 // ClearPanels() - вызываем у каждой панели очистку интерфейса,
98 // устанавливаем все наши поля на null/false.
99
100 // Метод не очищает панели как объекты, поэтому отписки пока не требую
101 // Панели по стандарту будет три. Для игры этого может быть достаточно
102 // Однако, логику можно будет слегка изменить, чтобы допустить возможн
103 // динамически, по ходу забега. В таком случае отписки следует добавит
104
105
106 private void ClickedPanel(ItemPanel panel)
107 {
108     if (panel == selectedPanel) { return; }
109     selectedPanel = panel;
110     continueButton.interactable = true;
111
112     foreach (var item in itemPanels)
113     {
114         if (selectedPanel == item)
115         {
116             selectedPanel.SetSelected(true);
117             selectedArtefactSO = selectedPanel.GetArtefact();
118         }
119         else item.SetSelected(false);
120     }
121 }
122
123 // ClickedPanel(ItemPanel panel) - реагирует на событие и получает пан
124 // по которой кликнули. Если панель и так выбрана - игнорируем.
125 // Если нет - выбираем панель, ищем среди листа панелей выбранную
126 // - Активируем и получаем её предмет.
127
128 }

```

⚠ Сейчас NextFloorScreen работает с панелями по ссылкам из инспектора. А количество предметов жестко выставлено в три. Для данного проекта этого достаточно и он не предполагает наличие большего количества. Однако, расширить систему можно очень легко, немного изменив NextFloorScreen, создавая нужное количество панелей или просто переиспользуя большее количество, чем 3. (Даже сейчас можно указать в лист больше панелей, создав их на сцене, просто

использоваться будет только 3. Можно будет в таком случае указать случайный дроп 2-5 предметов на выбор, но для меня это не нужно).

ArtefactsManager.

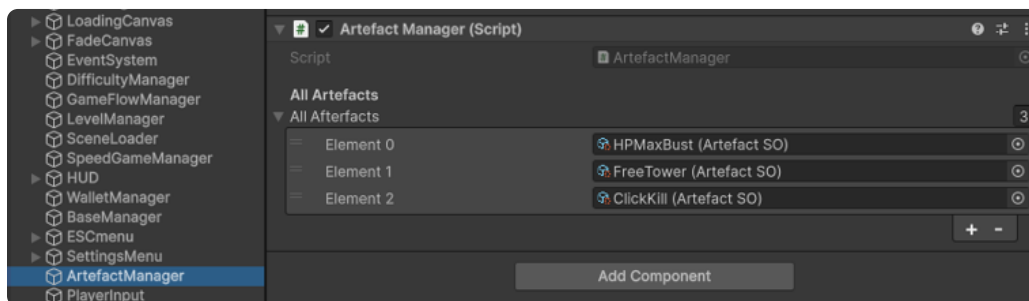
- ❗ Нам нужен некий менеджер, который будет хранить все артефакты игры и выдавать случайные по запросу. Он же послужит своеобразным инвентарем.

```
1 using NUnit.Framework;
2 using System.Collections.Generic;
3 using System.Linq;
4 using UnityEngine;
5
6 public class ArtefactManager : Manager<ArtefactManager>
7 {
8     [Header("All Artefacts")]
9     [SerializeField] private List<ArtefactSO> allArtefacts;
10
11     // Храним лист ArtefactSO - все доступные предметы.
12
13     private HashSet<ArtefactSO> equippedArtefacts = new();
14
15     // HashSet<ArtefactSO> - храним все надетые артефакты.
16
17     // GetRandomArtefacts(int) - сверяем все и те, что уже надеты, собираем
18     // из доступных лист и выкидываем наружу.
19
20     public List<ArtefactSO> GetRandomArtefacts(int count)
21     {
22         List<ArtefactSO> available = new();
23         foreach (var artefact in allArtefacts)
24         {
25             if (!equippedArtefacts.Contains(artefact))
26             {
27                 available.Add(artefact);
28             }
29         }
30
31         List<ArtefactSO> result = new();
32
33         count = Mathf.Min(count, available.Count);
34
35         while(result.Count < count)
36         {
37             int index = Random.Range(0, available.Count);
38             result.Add(available[index]);
39             available.RemoveAt(index);
40         }
41
42         return result;
43     }
44
45     public void EquipArtefact(ArtefactSO artefact)
46     {
47         if (!equippedArtefacts.Contains(artefact))
48         {
49             equippedArtefacts.Add(artefact);
50             artefact.effectArtefact.Equip();
51         }
52     }
53 }
```

```

54 // EquipArtefact(ArtefactSO) - принимает артефакт, запоминает, что он н
55 // а также вызывает абстрактный метод экипировки, что эффект предмета н
56 // действовать.
57
58 public void DequipArtefact(ArtefactSO artefact)
59 {
60     if (equippedArtefacts.Contains(artefact))
61     {
62         equippedArtefacts.Remove(artefact);
63         artefact.effectArtefact.Dequip();
64     }
65 }
66
67 // DequipArtefact(ArtefactSO) - отключаем эффект, достаём из хэш сета о
68
69 public void ResetInventory()
70 {
71     foreach (var item in equippedArtefacts.ToList())
72     {
73         DequipArtefact(item);
74     }
75     equippedArtefacts.Clear();
76 }
77 }

```



Ручное хранение всех артефактов.

Addressables.

Лично мне, в случае одиночной разработки пет проекта хватило бы и обычной системы ручного хранения, но хорошо бы понимать, как обстоят дела у серьёзных людей. А лучше даже внедрить!

В небольших проектах многие зависимости можно настроить вручную через Inspector. Однако при росте количества ресурсов становится сложнее контролировать их загрузку, зависимости и использование памяти.

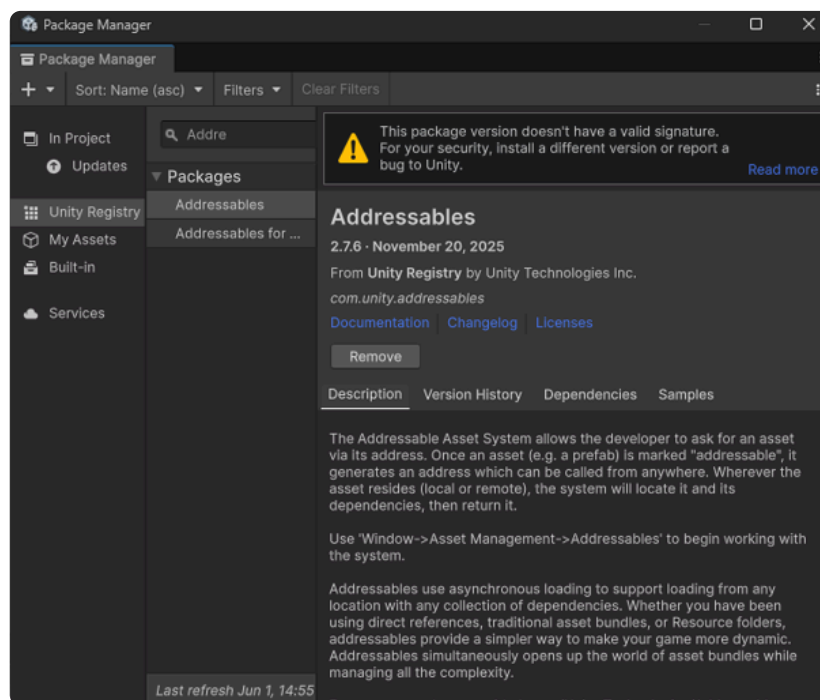
Когда речь идёт о большом количестве SO, префабов, моделей, текстур и других ресурсов, обычно используют специализированные механизмы загрузки ресурсов.

Одним из самых старых решений является папка Resources. Например, можно поместить все ArtefactSO в каталог: Assets/Resources/Artefacts/

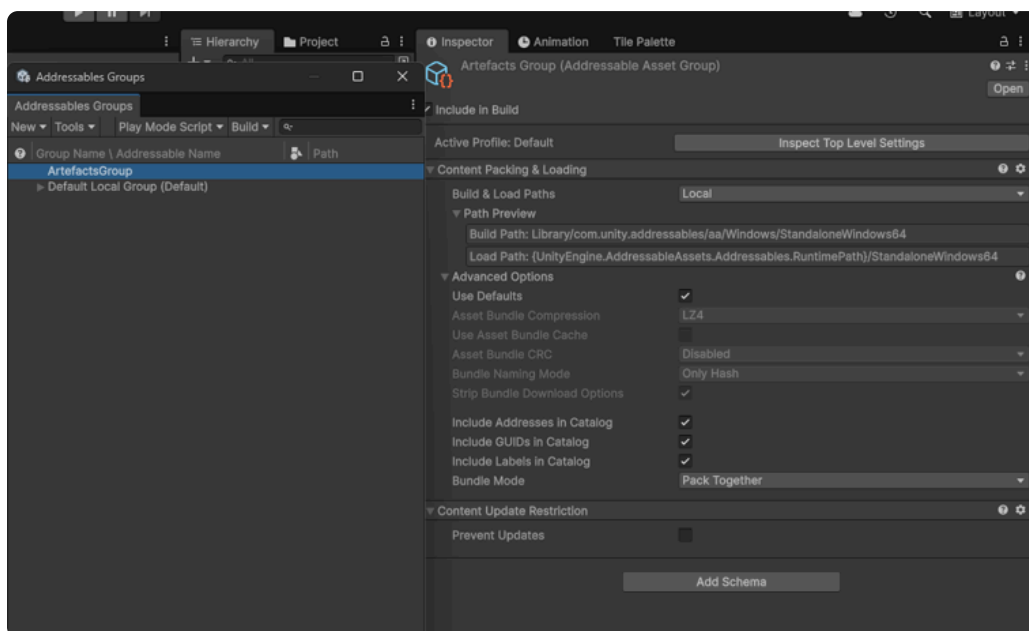
А затем просто подгружать следующим образом: `allAfterfacts = Resources.LoadAll<ArtefactSO>("Artefacts").ToList();`

Однако содержимое Resources всегда включается в билд. Кроме того, при запуске проекта Unity создаёт индекс всех ресурсов из этой папки, что увеличивает длительность загрузки и потребление памяти.

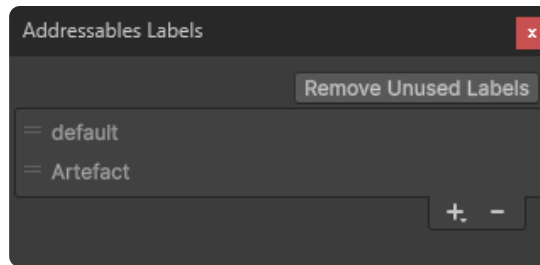
- ✓ Более современным решением является система Addressables, позволяющая адресно загружать и выгружать ресурсы по мере необходимости.



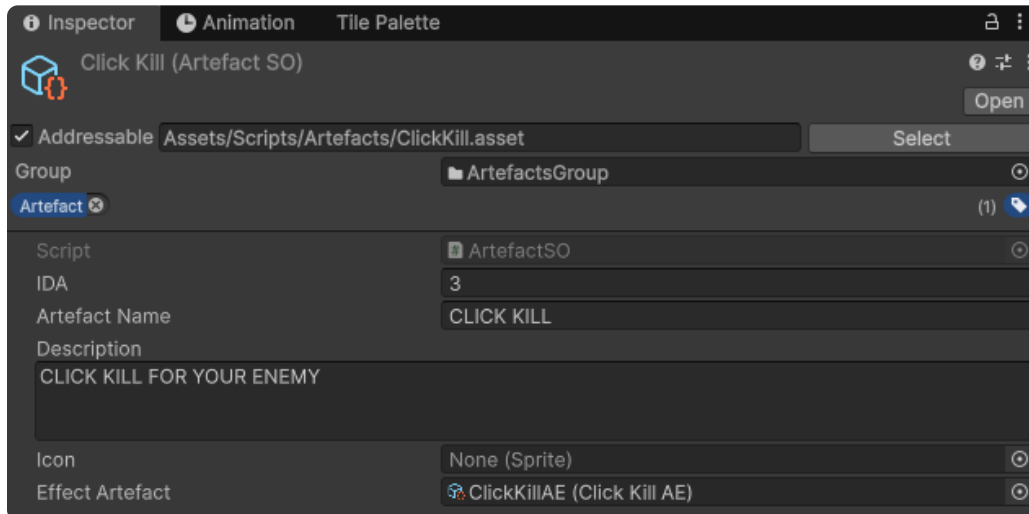
Устанавливаем из менеджера пакетов.



Создаём свою группу для артефактов.



Создаём специальную метку.



Устанавливаем галочку Addressable, группу и метку.

Теперь меняем ручное заполнение на следующий вариант:

```

1 using UnityEngine.AddressableAssets;
2
3 public class ArtefactManager : Manager<ArtefactManager>
4 {
5     private List<ArtefactSO> allArtefacts = new();
6     private HashSet<ArtefactSO> equippedArtefacts = new();
7
8     protected override void Awake()
9     {
10         base.Awake();
11         InitializeAllArtefacts();
12         if (allArtefacts.Count < 1)
13             Debug.LogError("Artefacts not detected!");
14     }
15
16     public async void InitializeAllArtefacts()
17     {
18         var handle = Addressables.LoadAssetsAsync<ArtefactSO>("Artefact
19         await handle.Task;
20         if (handle.Status == UnityEngine.ResourceManagement.AsyncOperat
21         {
22             allArtefacts = new List<ArtefactSO>(handle.Result);
23         }
24     }
25     /*
26     Подключаем пространство имён. Создаём асинхронную инициализацию.
27     Получаем операцию загрузки ассетов по типу ArtefactSO, передав туда мет
28     в поле handle. Ожидаем когда оно выполнится. После проверяем статус гот
29     и заполняем полученными результатами операции наш лист. Готово!
30     */

```

- ✔ Addressables позволяет работать на оптимизацию. Он даёт контроль над памятью, зависимостями и моментами загрузки ресурсов.

При использовании менеджером прямых ссылок на ресурсы, Unity вынужден учитывать все зависимости этих объектов. Прямая ссылка сама по себе не означает, что всё будет инстанцировано в сцене, но гарантирует, что Unity будет учитывать их при построении зависимостей и упаковке билда.

При Resources - Unity создаёт индекс всех ресурсов, находящихся в папке. При запуске игры движок должен построить индекс всех объектов, что увеличивает время старта и потребление памяти. При Resources.Load(...) ресурс загружается в память и становится доступным. Resources.UnloadUnusedAssets() - должен удалить объект, но он сможет это сделать лишь в случае, если на объект не осталось ссылок. В противном случае объект останется в памяти, так как на него всё ещё существуют ссылки.

Addressables заменяет прямые ссылки на косвенные (AssetReferences). AssetReference хранит GUID и информацию о расположении ресурса. Например, мы храним некоторые объекты локации отдельной меткой. На экране загрузки на эту локацию мы вызовем `await Addressables.LoadAssetsAsync<GameObject>("City2_Content", null).Task;` При такой загрузке по метке Addressables самостоятельно определяет, какие AssetBundle необходимо загрузить. Когда же игрок покидает локацию и переходит на новую, мы можем выгрузить данные из памяти: `Addressables.Release(city2Handle);` Однако и это не гарантирует немедленную выгрузку. Команда Release уменьшает счётчик ссылок на загруженный ресурс. Когда же он перестанет кем-либо использоваться, то Addressables сможет выгрузить его из памяти.

- ☐ Addressables строится поверх AssetBundle. Во время билда Addressables автоматически собирает помеченные ресурсы в AssetBundle (.bundle) и создаёт каталог адресов для их поиска и загрузки. Addressables строится поверх AssetBundle и автоматизирует работу с ними. Например, DLC может быть собрано в отдельный AssetBundle и загружаться с сервера только после покупки или активации дополнения.

Update:

- ☐ Представим, что игру сломали и теперь можно пройти больше этажей, чем я мог рассчитывать возможным. Можно добавить панель YOU WIN, в качестве уважения.

Однако игра всё-таки на рекорд, поэтому заканчивать игру совсем - не хочется. Предметы рано или поздно закончатся, и игра сломается. На такой случай я добавлю список не слишком оригинальных предметов, которые немного повышают характеристики, но которые могут складываться и попадаться игроку, даже когда уже попадались.